

Universidad del Valle de Guatemala

Deep Learning

Sección 20

Proyecto 2: Competencia de Agentes en Space Invaders

Diego Fabián Morales Dávila

25 de Octubre del 2026

1. Definición del problema y análisis del entorno

El objetivo es entrenar un agente de aprendizaje por refuerzo que maximice el puntaje de Space Invaders en el entorno ALE/SpaceInvaders-v5 de Gymnasium. El jugador controla un cañón en la franja inferior y dispara contra una formación de 36 invasores, en seis filas de seis, que avanza de lado a lado y desciende hacia el suelo. Cada invasor vale entre 5 y 30 puntos según su fila, y una nave nodriza que cruza la parte superior vale 200. La partida comienza con 3 vidas. La bandera terminated se activa al agotarse las vidas o cuando la formación toca el suelo, y la bandera truncated al llegar a 108000 fotogramas, equivalentes a 27000 decisiones del agente.

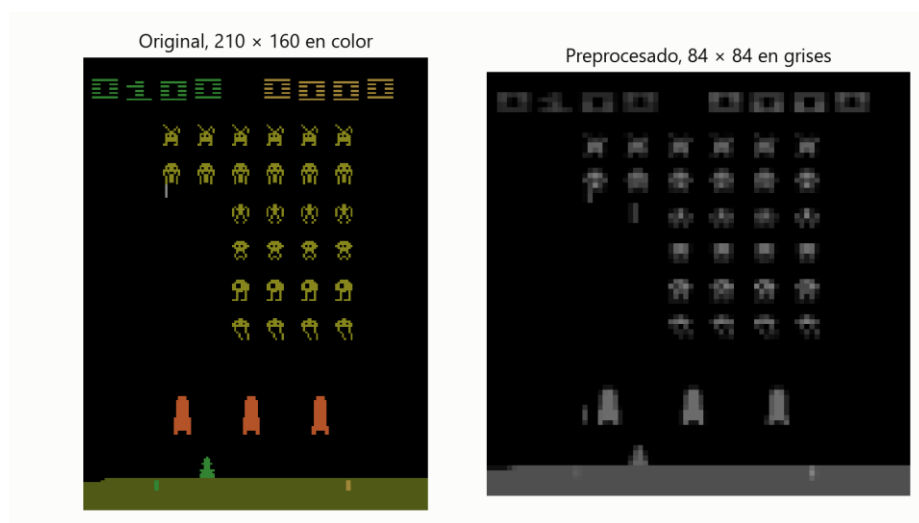


Figura 1. Fotograma original y preprocesado.

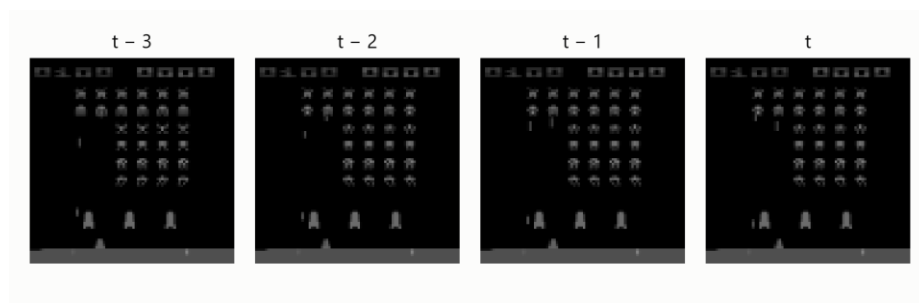


Figura 2. Pila de cuatro fotogramas.

El espacio de acciones es Discrete con 6 valores. Se usó el conjunto mínimo y no el completo de 18, porque ALE define el conjunto mínimo como las acciones que el juego distingue y las 12 restantes duplican el efecto de alguna de estas seis.

Índice	Identificador	Efecto
0	NOOP	No hace nada
1	FIRE	Dispara
2	RIGHT	Se mueve a la derecha
3	LEFT	Se mueve a la izquierda
4	RIGHTFIRE	Se mueve a la derecha y dispara
5	LEFTFIRE	Se mueve a la izquierda y dispara

Tabla 1. Espacio de acciones del entorno.

La versión v5 activa por defecto las acciones persistentes con probabilidad 0.25 en uno de cada cuatro pasos el emulador repite la acción anterior en lugar de la elegida, y cada decisión del agente se mantiene durante 4 fotogramas. Por eso se descartó aprender de una partida asistida por herramientas una secuencia calculada fotograma a fotograma no puede reproducirse aquí. El entorno vectorizado de ale-py trae la persistencia en 0 por defecto, y se fijó en 0.25 para no entrenar en un juego determinista distinto del evaluado.

La recompensa nativa coincide con el incremento del marcador. Con una política aleatoria, 40 episodios produjeron 382 eventos de recompensa. Su frecuencia decrece con el valor, porque los disparos desde abajo alcanzan primero a las filas inferiores, que son las que menos valen, y la nave nodriza apareció solo 2 veces.

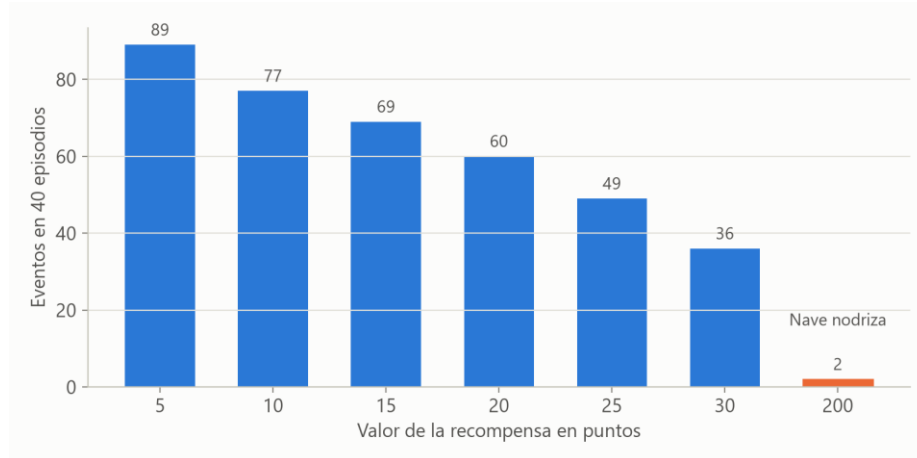


Figura 3. Valores de recompensa observados.

La señal no es dispersa. Con acciones aleatorias el 1.84 por ciento de los pasos recibe recompensa, una cada 48.5 pasos, y el agente final eleva esa proporción a 7.87 por ciento, una cada 12.6 pasos, dentro del horizonte de unos 100 pasos de un descuento de 0.99. Por eso no se diseñó ninguna recompensa adicional. Los valores sí requieren normalización, pues van de 5 a 200 como es estándar en DQN, se recortaron al signo al entrenar, y el puntaje real se registra aparte como única métrica reportada.

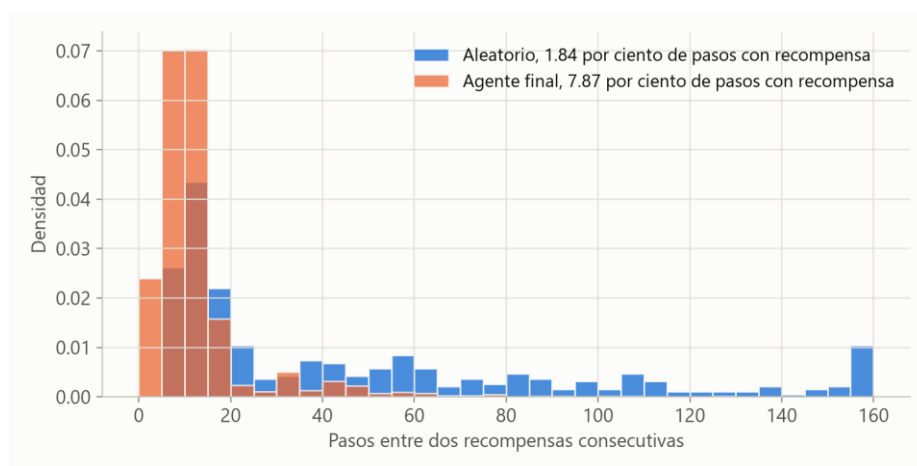


Figura 4. Densidad de la recompensa.

Como referencia se evaluaron dos políticas sin aprendizaje durante 30 episodios. La política aleatoria obtuvo 157.7 puntos de media y 515 de máximo. Un agente de reglas del laboratorio 5, que localiza por color al cañón y al invasor más cercano y esquiva los proyectiles próximos, obtuvo 272.2 de media y 540 de máximo.

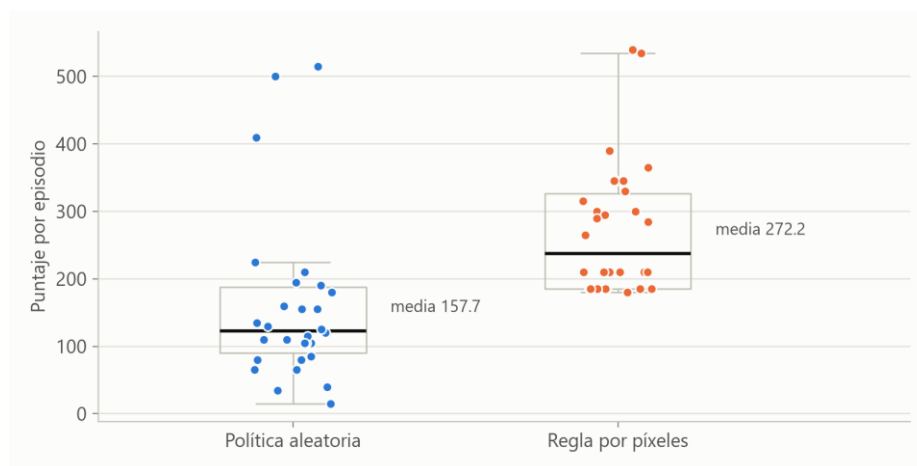


Figura 5. Puntaje de las políticas base.

Un solo entorno ejecuta 3288 pasos por segundo, y el entorno vectorizado nativo de ale-py llega a 14457 con 64 instancias en paralelo, suficiente para simular 50 millones de pasos en menos de una hora. El límite real del entrenamiento fue la memoria de repetición y la GPU, no el emulador.

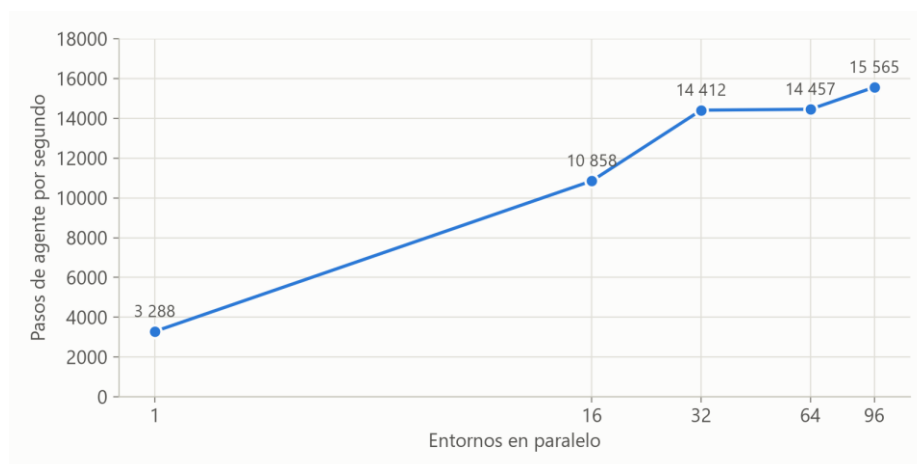


Figura 6. Rendimiento del entorno vectorizado.

Entrenar con el entorno vectorizado y evaluar con gym.make crea dos caminos que deben producir observaciones idénticas. Dos diferencias lo impedían, el disparo automático al reiniciar y el relleno inicial de la pila de fotogramas. Tras corregirlas, 400 de 400 observaciones coincidieron bit a bit, y esa comparación quedó como prueba automática.

Decisión	Justificación
Conjunto mínimo de 6 acciones	Las 12 restantes no tienen efecto distinto
Persistencia de 0.25 al entrenar	Es la configuración de la evaluación
Grises a 84 por 84	Conserva cañón, invasores y proyectiles
Máximo entre dos fotogramas	Los proyectiles parpadean
Pila de 4 fotogramas	Aporta dirección y velocidad
Hasta 30 acciones nulas al inicio	Diversifica los estados iniciales
Sin disparo automático al reiniciar	Rompía la equivalencia entre caminos
Vidas sin cortar el episodio	ALE no distingue pérdida de vida y final
Recompensa recortada al signo	Acota la escala del objetivo
Sin recompensa adicional	La señal nativa ya es densa

Tabla 2. Decisiones de diseño del entorno.

2. Metodología de desarrollo

Se consideraron DQN, Double DQN, Dueling DQN, los retornos de n pasos, la repetición priorizada, IQN y PPO. Se eligió la familia DQN porque, con la simulación casi gratuita, lo que limita es cuánto aprende el agente de cada transición, y un método fuera de política reutiliza cada una varias veces, además sus extensiones se pueden activar una por una. Todo se implementó desde cero en PyTorch sobre un único bucle de entrenamiento, de modo que las variantes solo difieren en el componente evaluado.

La red es la convolucional de Mnih y colaboradores tres capas de 32, 64 y 64 filtros, con núcleos de 8, 4 y 3 píxeles y pasos de 4, 2 y 1, que reducen la entrada de 4 por 84 por 84 a 64 mapas de 7 por 7, es decir 3136 características. En la variante Dueling estas alimentan dos ramas de 512 unidades, una para el valor del estado y otra para la ventaja de cada acción, que se combinan restando la media de las ventajas. La normalización de los píxeles ocurre dentro de la red para que no pueda olvidarse al evaluar.

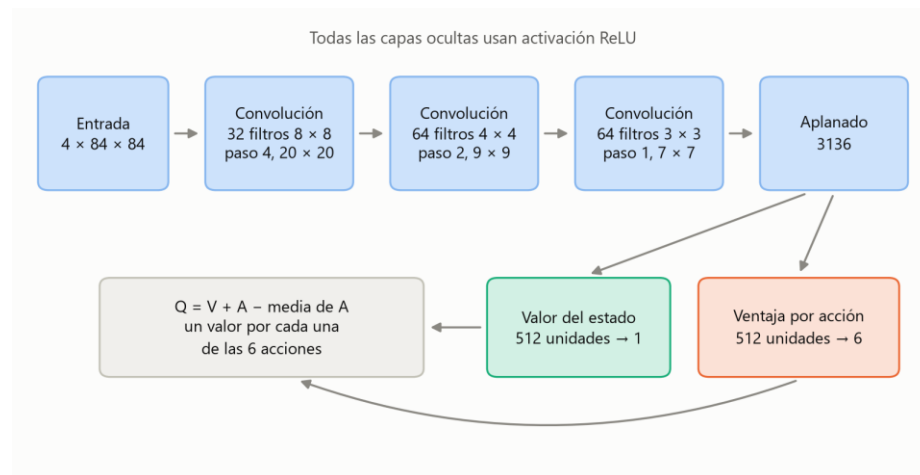


Figura 7. Arquitectura de la red.

Hiperparámetro	Valor
Algoritmo	DQN con Double DQN y Dueling
Retornos	3 pasos
Factor de descuento	0.99
Pérdida	Huber
Optimizador	Adam
Tasa de aprendizaje	0.00025
Lote	512
Memoria de repetición	750 mil transiciones
Inicio del aprendizaje	80 mil pasos
Red objetivo	Copia cada 40 mil pasos
Recorte de gradiente	Norma 10
Entornos en paralelo	64
Pasos de entrenamiento	50 millones

Tabla 3. Hiperparámetros del modelo final.

La exploración siguió una política épsilon voraz. Los primeros 80 mil pasos son aleatorios para llenar la memoria, y después épsilon descende linealmente de 1.0 a 0.01 durante el primer millón

de pasos. Como el entorno ya introduce aleatoriedad con la persistencia de acciones, la exploración residual necesaria es baja.

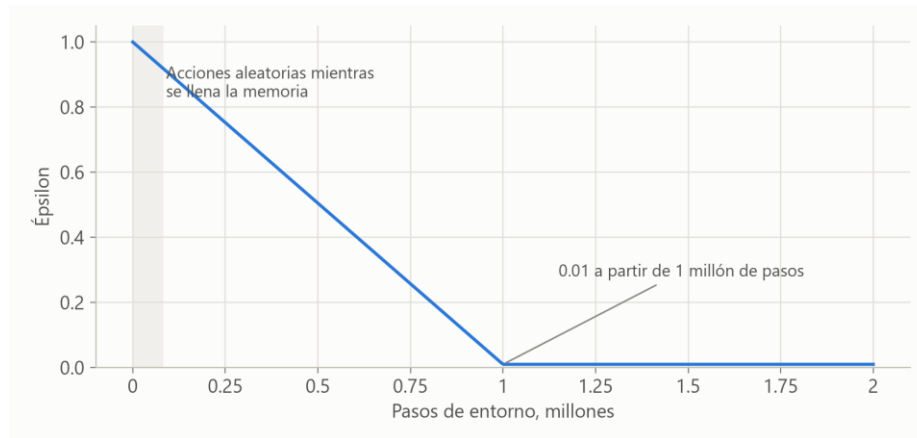


Figura 8. Calendario de exploración.

Cada paso vectorizado produce 64 transiciones y se hace una actualización con un lote de 512, la misma razón de 8 muestras por transición del DQN original. La memoria de repetición guarda un solo fotograma por transición y reconstruye las pilas al muestrear, porque dos pasos consecutivos comparten 3 de sus 4 fotogramas un millón de transiciones ocupa 6.6 GB en lugar de 26.3 GB. Una prueba automática verifica que la reconstrucción es exacta.

Un perfil del paso de entrenamiento mostró que el muestreo de la memoria consumía el 57 por ciento del tiempo. Reunir los fotogramas del lote en una sola operación de indexado bajó ese costo de 12.96 a 4.31 milisegundos en una prueba aislada, y un hilo que prepara el siguiente lote durante la actualización lo ocultó casi por completo. La precisión mixta se descartó bf16 tardaba 4.11 milisegundos por actualización contra 3.39 en precisión simple. El rendimiento pasó de 2150 a unos 5000 pasos por segundo.

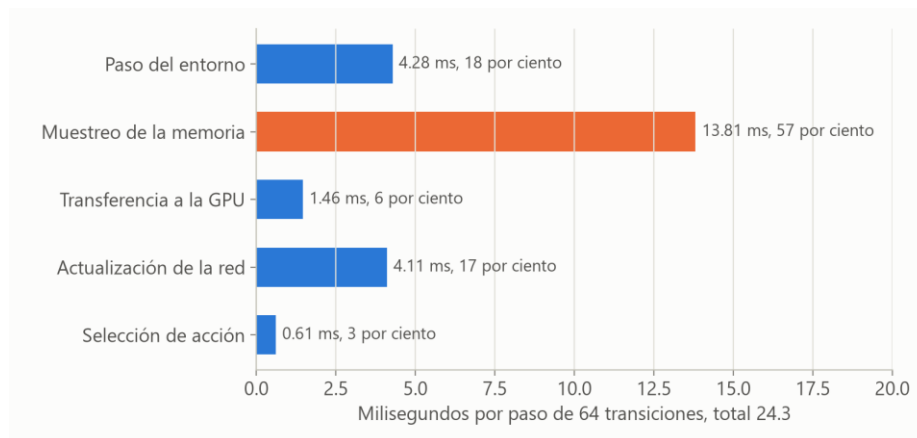


Figura 9. Costo del paso de entrenamiento.

La evaluación usa siempre el puntaje real. Como la competencia califica el mayor de 5 episodios y no el promedio, además de la media se estimó por remuestreo el máximo esperado de 5 episodios.

3. Resultados de las iteraciones

Cada iteración agrega un solo componente sobre la anterior, con la misma semilla y los mismos hiperparámetros, para atribuir cada diferencia a su causa. Las comparaciones se rehicieron al final con 100 episodios por modelo y semillas idénticas, y con 200 en las dos últimas filas, porque las evaluaciones periódicas del entrenamiento usan menos episodios. La columna de entrenamiento es la media de los últimos 200 episodios.

It.	Cambio	Pasos	Entrenamiento	Evaluación	Máximo de 5
0	Política aleatoria			157.7	308.6
1	Reglas por píxeles			272.2	395.2
2	DQN base	4 M	511.0	494.1	686.8
3	Double DQN	4 M	530.5	534.2	676.0
4	Dueling	4 M	574.7	612.0	775.2
5	Retornos de 3 pasos	4 M	875.8	904.5	1302.6
6	Repetición priorizada	4 M	702.4	752.9	1029.5
7	IQN sobre la iteración 5	4 M	565.1	578.4	731.4
8	Iteración 5 prolongada	20 M	1898.8	2024.2	2865.0
9	Sin recorte de recompensa	20 M	1036.5	981.4	1651.2
10	Descuento de 0.997	20 M	1610.3	1816.2	2545.1
11	Iteración 8 hasta 50 M	50 M	2342.2	2379.0	2899.4
12	Selección de punto de control	50 M	2342.2	2563.1	2921.2

Tabla 4. Historial completo de iteraciones.

El DQN base ya evaluaba 282 puntos a los 500 mil pasos, por encima del agente de reglas, y llegó a 494.1 de media a los 4 millones. Double DQN aportó poco, 534.2, y Dueling algo más, 612.0. El salto lo dieron los retornos de 3 pasos 904.5 de media y 1302.6 de máximo esperado, un 48 por ciento sobre la iteración anterior.

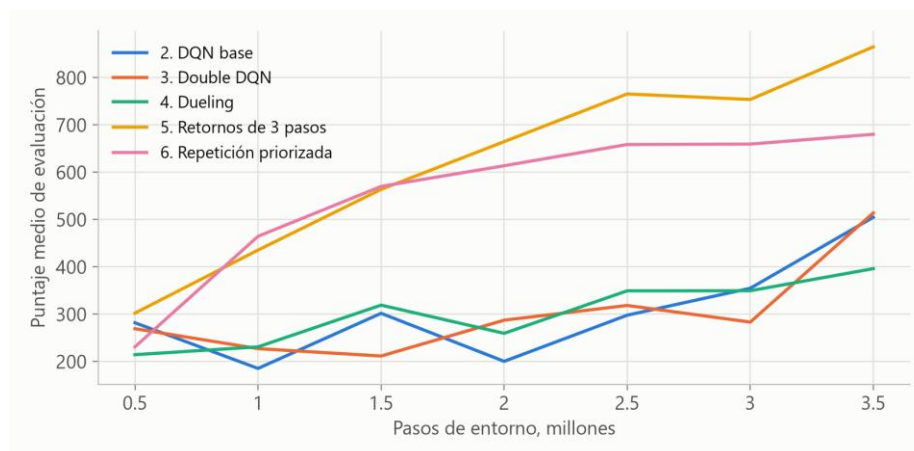


Figura 10. Evaluación de las ablaciones.

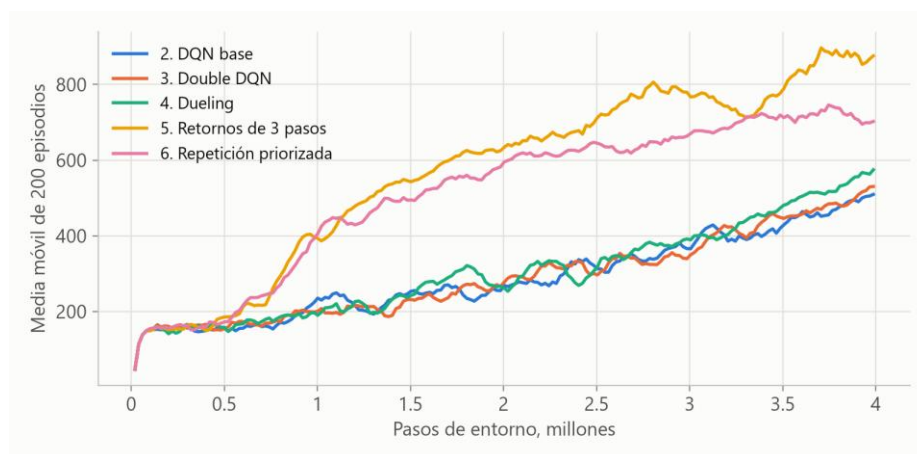


Figura 11. Puntaje de entrenamiento por ablación.

Con retornos de un paso, la recompensa de un disparo necesita muchas actualizaciones para retroceder hasta la decisión que lo originó con 3 pasos llega en una sola. Así, el valor Q medio alcanza 4 a los 4 millones de pasos, un nivel que el DQN base alcanza a los 10 millones. Ningún entrenamiento mostró divergencia de los valores Q. Sí hubo un estancamiento temprano en una corrida de prueba a los 100 mil pasos la política voraz obtuvo 6.9 puntos contra 146 con exploración, porque una acción dominaba por márgenes mínimos, y se resolvió al seguir entrenando.

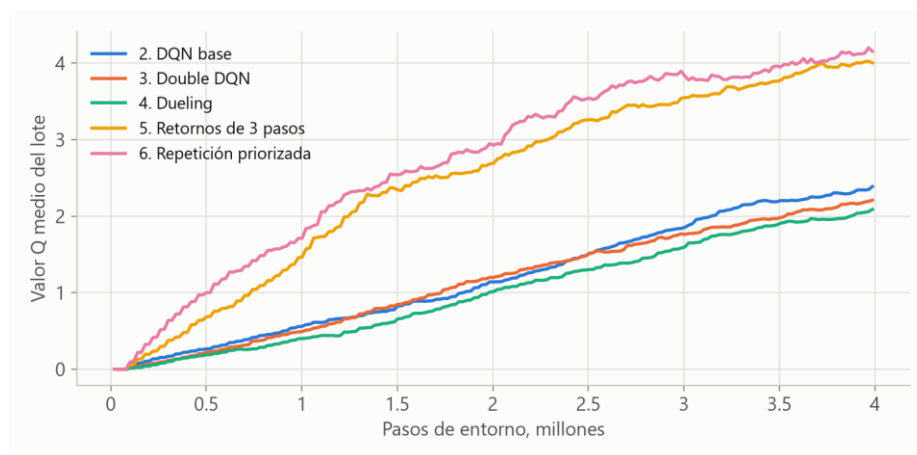


Figura 12. Evolución del valor Q.

La repetición priorizada bajó la media a 752.9, un 17 por ciento menos. Una explicación consistente con los datos es que con 64 entornos en paralelo la memoria ya es muy diversa, y priorizar concentra el muestreo en pocas transiciones con error alto. Por eso se excluyó del modelo final.

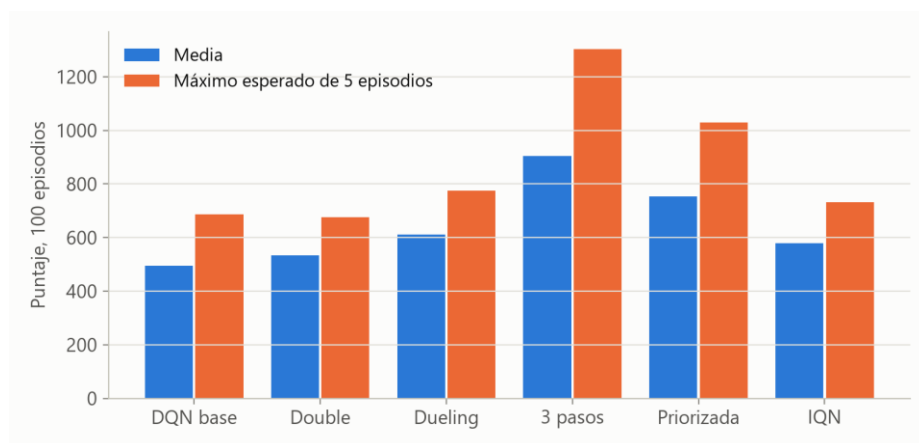


Figura 13. Comparación a cuatro millones.

En la iteración 5, un ϵ de evaluación de 0.01 subió el máximo esperado de 1308.5 a 1384.0 frente a la política voraz pura. En el modelo final ese beneficio desapareció la política voraz dio el mejor valor, 2941.6, porque la persistencia de acciones ya aporta la variabilidad necesaria.

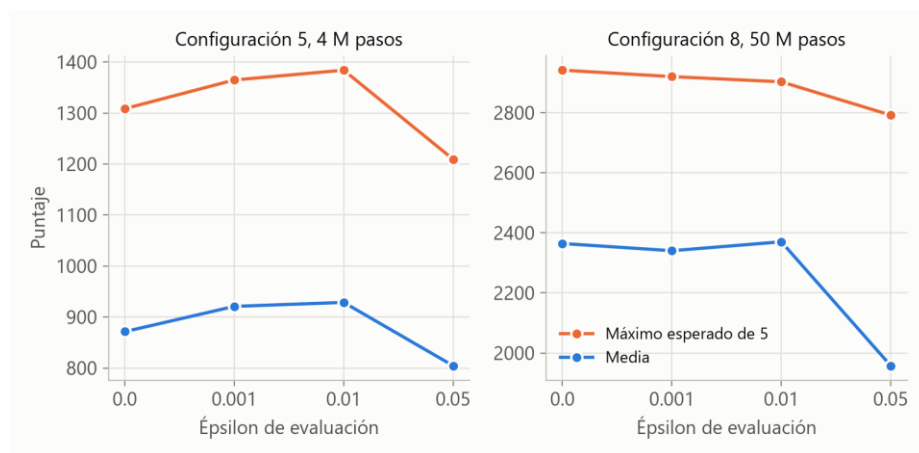


Figura 14. Barrido de ϵ .

IQN, que en los resultados publicados supera ampliamente a DQN en este juego, rindió por debajo con el presupuesto disponible 578.4 a los 4 millones de pasos, frente a 904.5 de la configuración equivalente. Su red cuesta cerca del doble por paso, y a los 11 millones apenas igualaba, con 910.0, lo que la iteración 5 logró en 4.

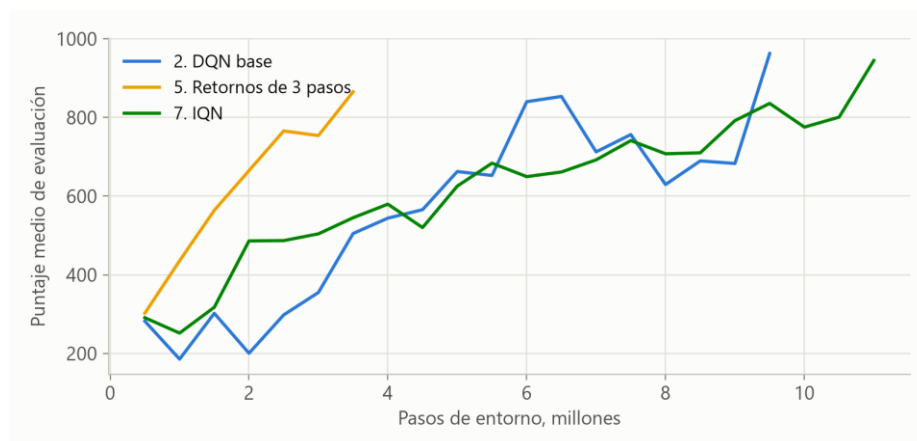


Figura 15. IQN frente a DQN.

Como IQN aprende la distribución de retornos, se puede evaluar con una distorsión de riesgo sin reentrenar. La hipótesis inicial era que buscar riesgo favorecería la métrica de máximo. Ocurrió lo contrario las distorsiones que buscan riesgo empeoraron la media y el máximo, y las que lo evitan mejoraron ambos, hasta 1473.0 de máximo esperado frente a 1336.6 de la valuación neutral.

Distorsión	Tipo	Media	Máximo de 5
Neutral	Neutral	910.0	1336.6
Cola superior, 75 por ciento	Busca riesgo	868.8	1331.3
Cola superior, 50 por ciento	Busca riesgo	849.5	1279.7
Cola superior, 25 por ciento	Busca riesgo	773.7	1172.0
Wang, 0.75	Busca riesgo	802.5	1230.6
Potencia, 1.0	Evita riesgo	1032.1	1473.0
Cola inferior, 50 por ciento	Evita riesgo	998.5	1426.8

Tabla 5. Distorsiones de riesgo en IQN.

La configuración de la iteración 5 se entrenó durante 50 millones de pasos, en 184 minutos a unos 5100 pasos por segundo. El puntaje crece con rapidez hasta cerca de 17 millones de pasos, y a partir de 25 millones avanza despacio y con oscilaciones de unos 300 puntos.

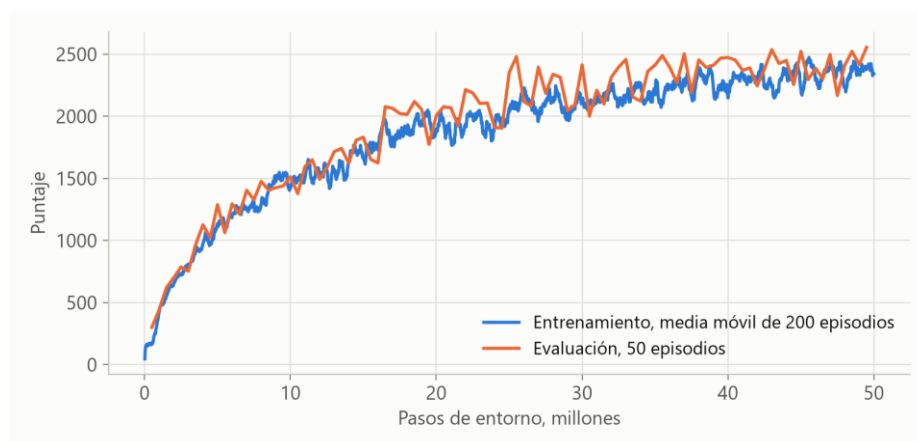


Figura 16. Entrenamiento largo del campeón.

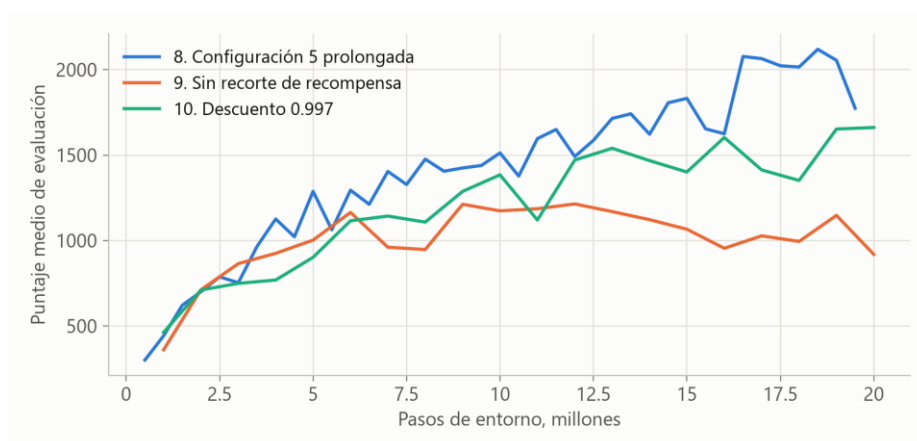


Figura 17. Recompensa y horizonte.

El punto de control guardado como el mejor se eligió con evaluaciones de 50 episodios y un error estándar de más de 100 puntos, y tomar el máximo de decenas de evaluaciones con ese ruido selecciona suerte. Al reevaluar 13 puntos de control con 200 episodios, el elegido quedó en el décimo lugar. El máximo esperado resultó casi igual en todos, entre 2887 y 2964, así que se eligió por media y varianza el punto final, con 2563.1 de media y desviación de 601.4, frente a 2379.0 y 741.9 del original.

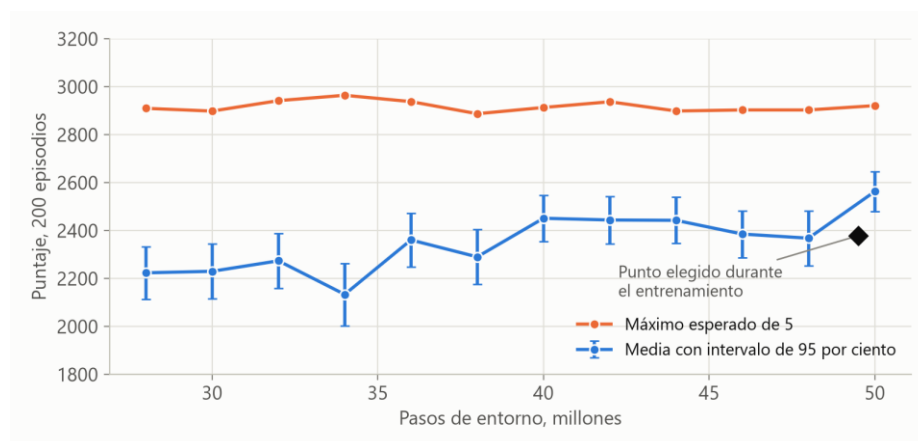


Figura 18. Selección de puntos de control.

4. Discusión de resultados

Ordenar los cambios por magnitud de efecto deja un patrón claro lo que más aportó fue entrenar más y propagar mejor el valor, y lo que más restó fue lo que empuja al agente a arriesgarse.

Cambio	Efecto sobre la media
Entrenar de 20 a 50 millones de pasos	+354.8
Retornos de 3 pasos	+292.5
Selección de punto de control	+184.1
Dueling	+77.8
Double DQN	+40.1
Repetición priorizada	-151.6
Descuento de 0.997	-208.0
IQN en lugar de DQN	-326.1
Quitar el recorte de recompensa	-1042.8

Tabla 6. Impacto de cada cambio.

En 250 episodios del modelo final, el puntaje correlaciona 0.91 con la duración del episodio Space Invaders es un juego de supervivencia antes que de puntería. Además, el 44 por ciento de los episodios termina exactamente en 2815 puntos, un techo demasiado repetible para ser azar.

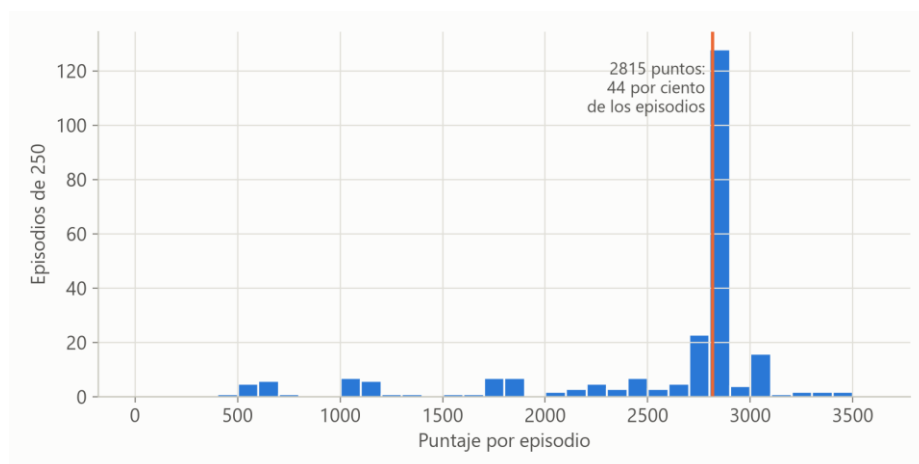


Figura 19. Distribución del puntaje final.

La causa de ese techo no es que el agente sea abatido. En 27 de 40 episodios la partida terminó con la pérdida simultánea de 2 o 3 vidas, lo que solo ocurre cuando la formación toca el suelo, y todos esos finales cayeron entre 2765 y 3020 puntos. Solo 13 episodios terminaron por un disparo sobre la última vida, con puntajes dispersos entre 470 y 3265. El agente limpia las primeras oleadas con regularidad, pero cuando la formación ya está baja y no quedan escudos, no alcanza a eliminarla antes de que aterrice.

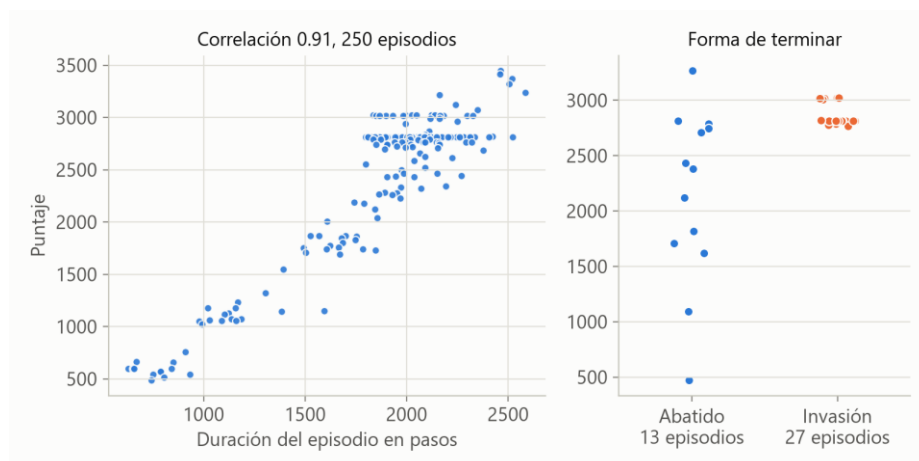


Figura 20. Puntaje según duración y final.

Esto ayuda a entender los experimentos fallidos. Sin recorte, el agente aprende que la nave nodriza vale 40 veces más que un invasor, y es plausible que perseguirla lo saque de posición a costa de vidas las distorsiones que buscan riesgo fallan por la misma razón. Visto así, el recorte de la recompensa no es una pérdida de información aplana justamente la tentación de perseguir premios grandes.

El video del mejor episodio, 3445 puntos en 2462 pasos, y el registro de 40 episodios muestran la estrategia aprendida. El 84 por ciento de las acciones incluye disparo y solo el 6.2 por ciento consiste en no hacer nada el agente se desplaza casi siempre disparando y pasa el 17 por ciento del tiempo en el extremo izquierdo de su recorrido. En el video despeja oleadas completas protegido por los escudos, y al final, sin ellos, la formación queda cada vez más baja.

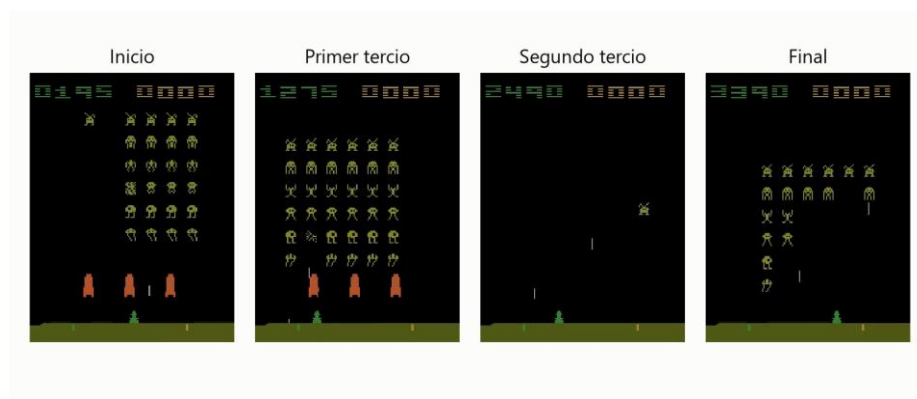


Figura 21. Fotogramas del mejor episodio.

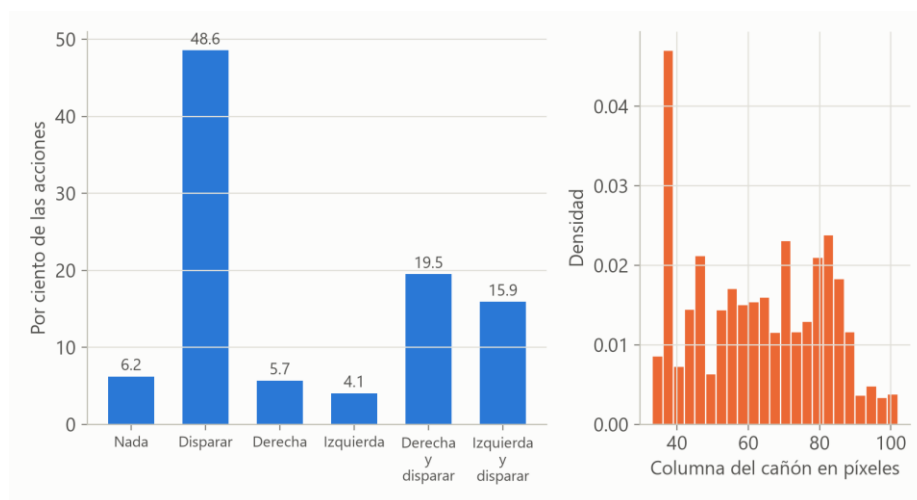


Figura 22. Acciones del agente campeón.

Las limitaciones son de cómputo y de alcance. Se usó una sola semilla, así que las diferencias pequeñas entre iteraciones, como la de Double DQN, están dentro de la variación esperable entre semillas. IQN no llegó a los cientos de millones de fotogramas de sus resultados publicados. Y cerca de 10 GB de RAM libre limitaron la memoria de repetición a 750 mil transiciones e impidieron correr dos experimentos grandes a la vez.

Sobre el equilibrio entre exploración y explotación, este entorno explora por sí mismo la persistencia de acciones basta para que un ϵ final de 0.01 funcione, y subirlo a 0.05 al evaluar bajó la media de 2364.5 a 1957.7. En el plano de la estrategia gana la explotación conservadora todo lo que empujó al agente hacia apuestas de alto valor lo perjudicó.

5. Conclusiones

- Los retornos de 3 pasos fueron el componente de mayor impacto.
- Hay una correlación 0.91 con la duración del episodio y todo cambio que empujó al agente a arriesgarse lo empeoró.
- Con puntajes tan dispersos, elegir modelos con pocas evaluaciones selecciona suerte.

Métrica	Valor
Media, 250 episodios	2508.1
Mediana	2815
Desviación estándar	663.0
Percentil 90	2963
Máximo esperado de 5	2940.3
Récord registrado en video	3445
Mayor de 5, protocolo de competencia	2815
Política aleatoria	157.7

Tabla 7. Desempeño final del agente.

Repositorio de GitHub

<https://github.com/FabianMoraeles/DeepProyecto2>